

Computer Graphics

Week 3: 2D Graphics (2)

Sub Topics

- Hierarchical Modeling
- HTML Canvas Graphics

Hierarchical Modeling Subtopics

- Building Complex Objects
- Scene Graphs
- The Transform Stack

Building Complex Objects

- The modeling transformations that are used to place an object in the scene should not affect other objects in the scene
- To limit their application to just the one object, we can save the current transformation before starting work on the object and restore it afterwards
  - How this is done differs from one graphics API to another, but let's suppose here that there are subroutines `saveTransform()` and `restoreTransform()` for performing those tasks
  - `saveTransform` will make a copy of the modeling transformation that is currently in effect and store that copy
  - Later, when `restoreTransform` is called, it will retrieve that copy and will replace the current modeling transform with the retrieved transform

Building Complex Objects

- Typical code for drawing an object will then have the form:

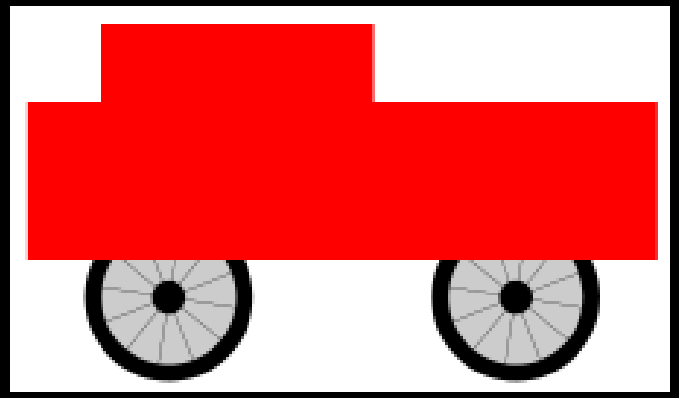
```
saveTransform()
translate(dx,dy) // move object into position
rotate(r)        // set the orientation of the object
scale(sx,sy)     // set the size of the object
      .
      .  // draw the object, using its natural coordinates
      .
restoreTransform()
```

- We don't know and don't need to know what the saved transform does.
- Perhaps it is simply the so-called identity transform, which is a transform that doesn't modify the coordinates to which it is applied
- The modeling transform moves the object from its natural coordinates into its proper place in the scene

Building Complex Objects

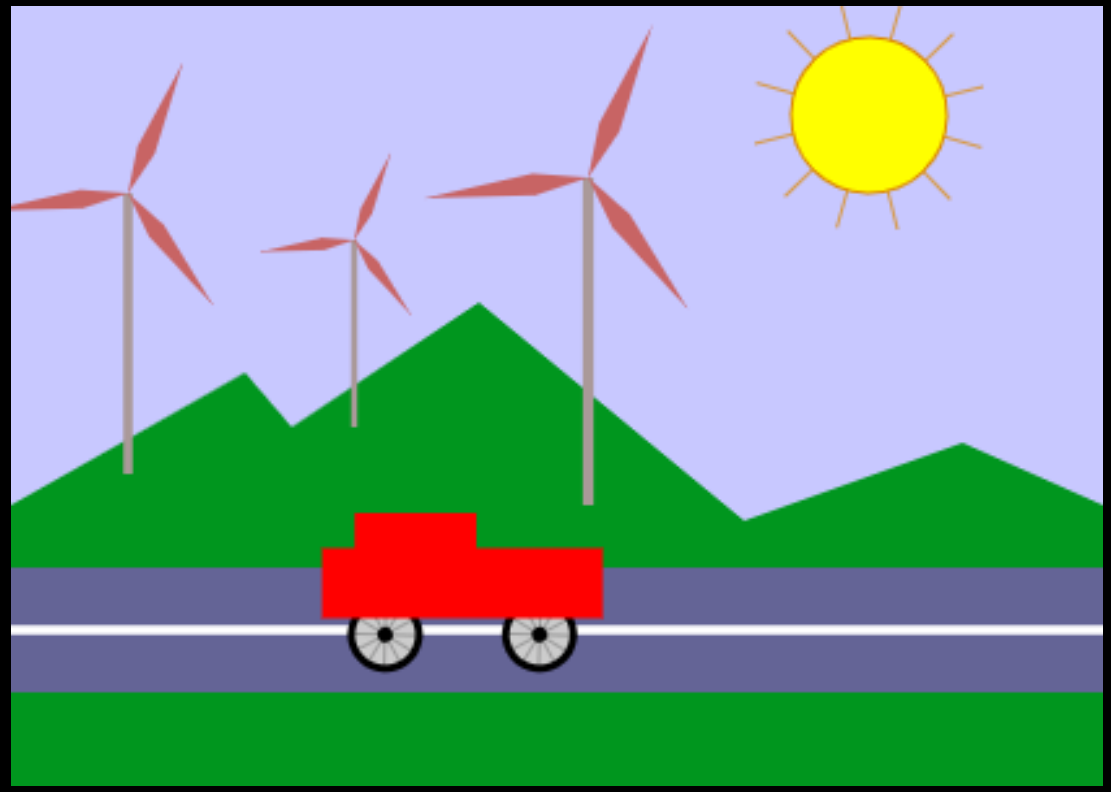
- We can build objects that are made up of smaller objects which in turn are made up of even smaller objects, to any level
- Suppose that we want to draw a simple 2D image of a cart with two wheels
- Here is pseudocode for a subroutine that draws the cart in its own coordinate system:

```
subroutine drawCart() :
    saveTransform()        // save the current transform
    translate(-1.65,-0.1) // center of first wheel will be at (-1.65,-0.1)
    scale(0.8,0.8)         // scale to reduce radius from 1 to 0.8
    drawWheel()            // draw the first wheel
    restoreTransform()     // restore the saved transform
    saveTransform()        // save it again
    translate(1.5,-0.1)    // center of second wheel will be at (1.5,-0.1)
    scale(0.8,0.8)         // scale to reduce radius from 1 to 0.8
    drawWheel(g2)          // draw the second wheel
    restoreTransform()     // restore the transform
    setDrawingColor(RED)   // use red color to draw the rectangles
    fillRectangle(-3, 0, 6, 2)      // draw the body of the cart
    fillRectangle(-2.3, 1, 2.6, 1)  // draw the top of the cart
```



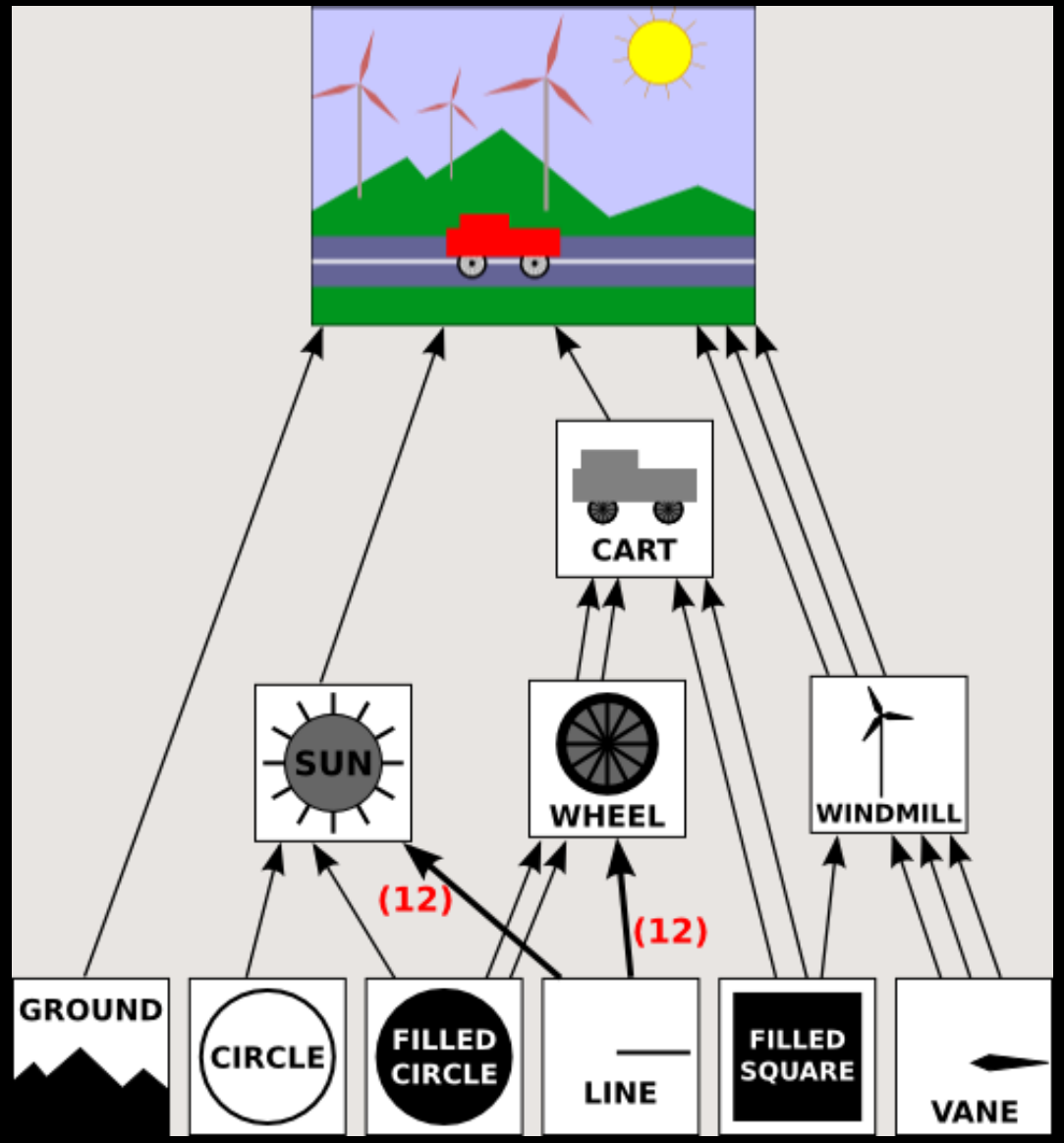
Building Complex Objects

- Sample animation demo



Scene Graphs

- Scene graph is a data structure that represents the objects in a scene, together with attributes of the objects and the modeling transformations that are applied to the objects
- A scene graph might exist only conceptually, or it might be an actual data structure in a program



Scene Graphs

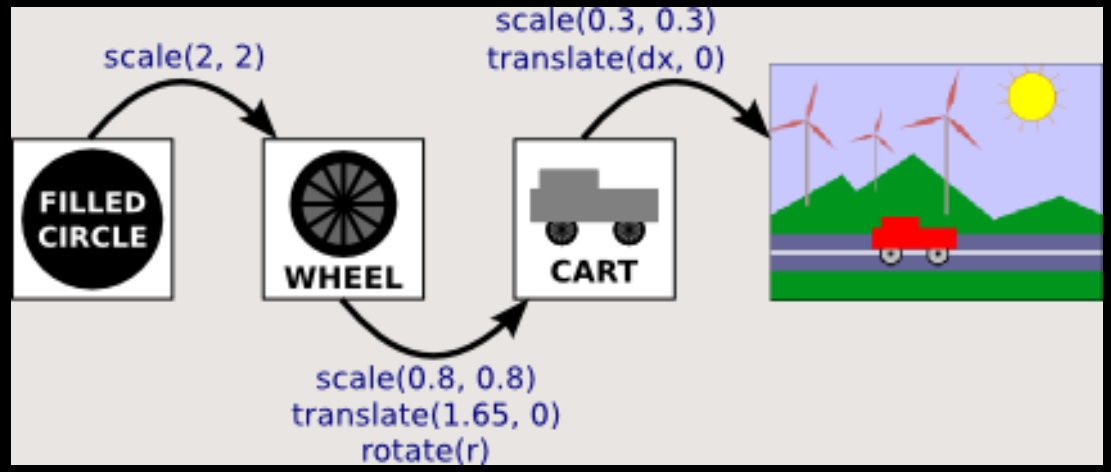
- When implementing a scene graph as a data structure made up of objects, a decision has to be made about how to handle transforms
  - One option is to allow transformations to be associated with any node in the scene graph
- Another decision has to be made about how to handle color
  - One option is that a color on a compound object acts as a default color for its sub-objects, but color can be overridden on the sub-objects

The Transform Stack

- When drawing a complex object, one subroutine can call other subroutines
  - This means that several drawing subroutines can be active at the same time, each with its own saved transform
  - When a transform is saved after another transform has already been saved, the system needs to remember both transforms
  - When `restoreTransform()` is called, it is the most recently saved transform that should be restored → need a **stack**
- Some graphics APIs come with transform stacks already defined
  - The original OpenGL API includes the functions `glPushMatrix()` and `glPopMatrix()` `for` using a stack of transformation matrices
  - The HTML canvas API for 2D graphics includes functions named `save()` and `restore()` that are actually push and pop operations on a stack

The Transform Stack

- Your responsibility is to design the individual objects, in their own natural coordinate system
  - specify the modeling transformations that are applied to the sub-objects of that object
- You construct the scene in a similar way
- The computer will then put everything together for you, considering the many layers of hierarchical structure
- That's the power of hierarchical design; that's how it helps you deal with complexity



HTML Canvas Graphics

The 2D Graphics Context; Shapes; Stroke and Fill; Trasforms; Aux Canvas; Pixel Manipulation; Images

HTML Canvas Graphics

- Most modern web browsers support a **2D graphics API** that can be used to create images on a web page
  - The API is implemented using **JavaScript**, the client-side programming language for the web
- HTML canvas: an HTML element that can be used as a drawing surface, created with

```
<canvas width="800" height="600" id="theCanvas"></canvas>
```

- To draw on a canvas, you need a graphics context: an object that contains functions for drawing shapes and variables that record the current graphics state, including the current drawing color, transform, and font

```
canvas = document.getElementById("theCanvas");
graphics = canvas.getContext("2d");
```

Sample HTML canvas graphics

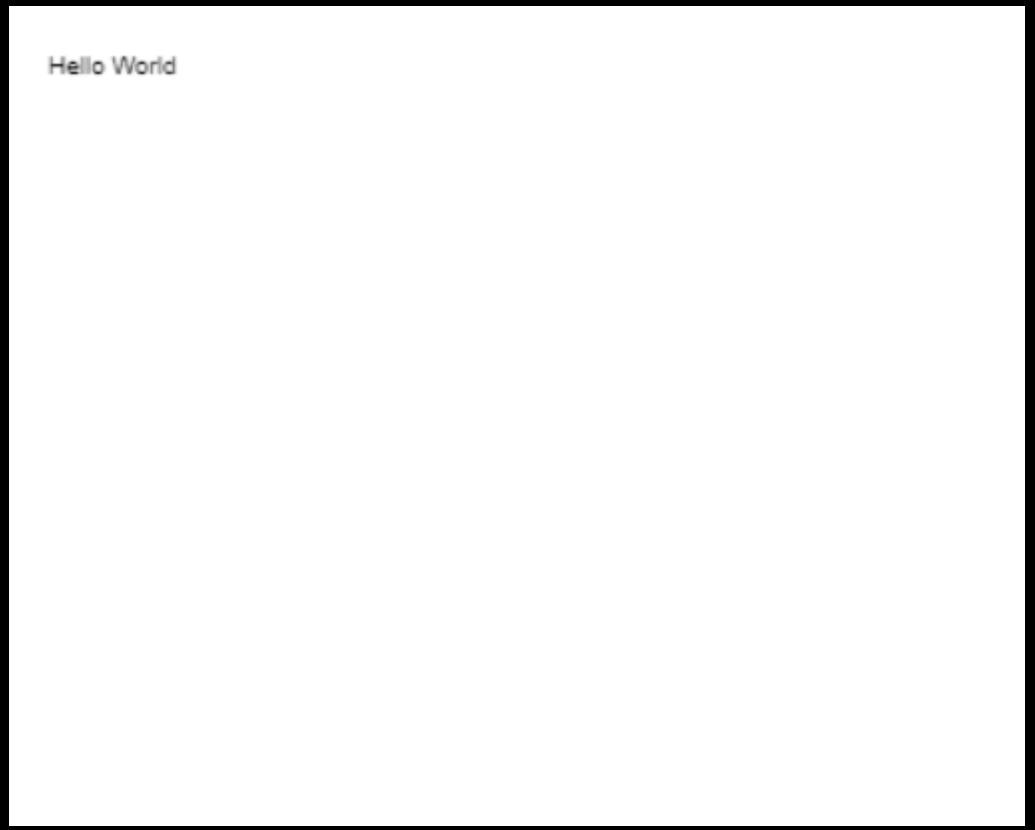
```
<!DOCTYPE html>
<html>
<head>
<title>Canvas Graphics</title>
<script>
    let canvas;    // DOM object corresponding to the canvas
    let graphics;  // 2D graphics context for drawing on the canvas

    function draw() {
           // draw on the canvas, using the graphics context
        graphics.fillText("Hello World", 10, 20);
    }

    function init() {
        canvas = document.getElementById("theCanvas");
        graphics = canvas.getContext("2d");
        draw();  // draw something on the canvas
    }

    window.onload = init;

</script>
</head>
<body>
    <canvas id="theCanvas" width="640" height="480"></canvas>
</body>
</html>
```



Shapes

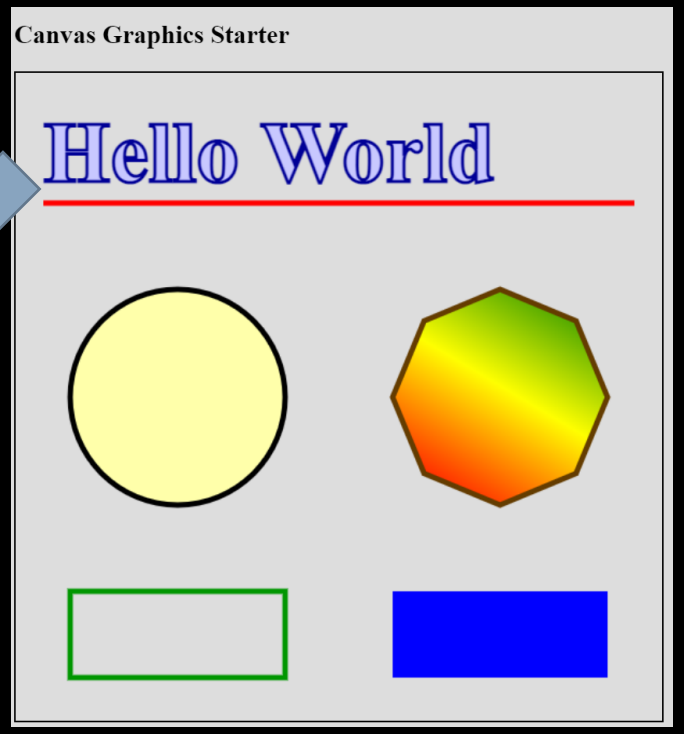
- The canvas coordinate system
  - Upper left corner: (0,0)
  - The x-coordinate increases to the right
  - The y-coordinate increases downward
- HTML canvas only supports rectangles and text as the basic shapes
- Other shapes must be created as paths

Shapes

- If *graphics* refers to the object that represents the graphics context:
  - graphics.fillRect(x,y,w,h) – draws a filled rectangle with corner at (x,y), with width w and with height h
  - graphics.strokeRect(x,y,w,h) – strokes the outline of the same rectangle.
  - graphics.clearRect(x,y,w,h) – clears the rectangle by filling it with fully transparent pixels, allowing the background of the canvas to show
  - graphics.fillText(str,x,y) – fills the characters in the string str; the left end of the baseline of the string is positioned at the point (x,y)
  - graphics.strokeText(str,x,y) – strokes the outlines of the characters in the string

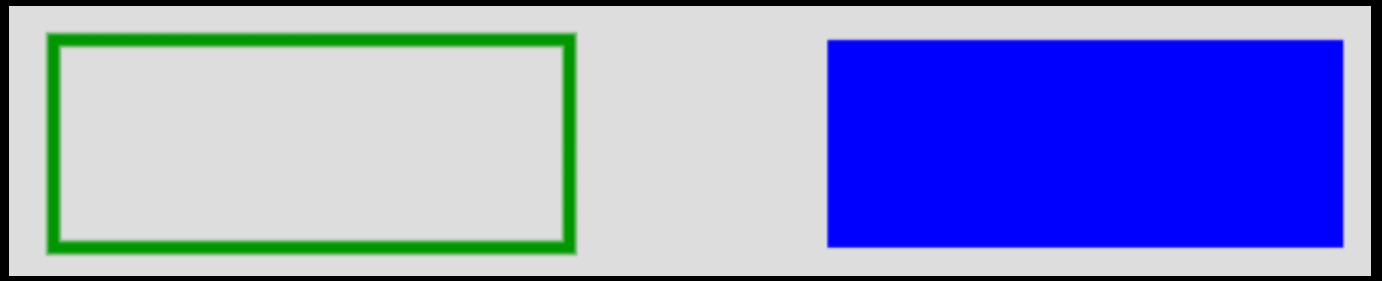
Sample Canvas 2D Graphics

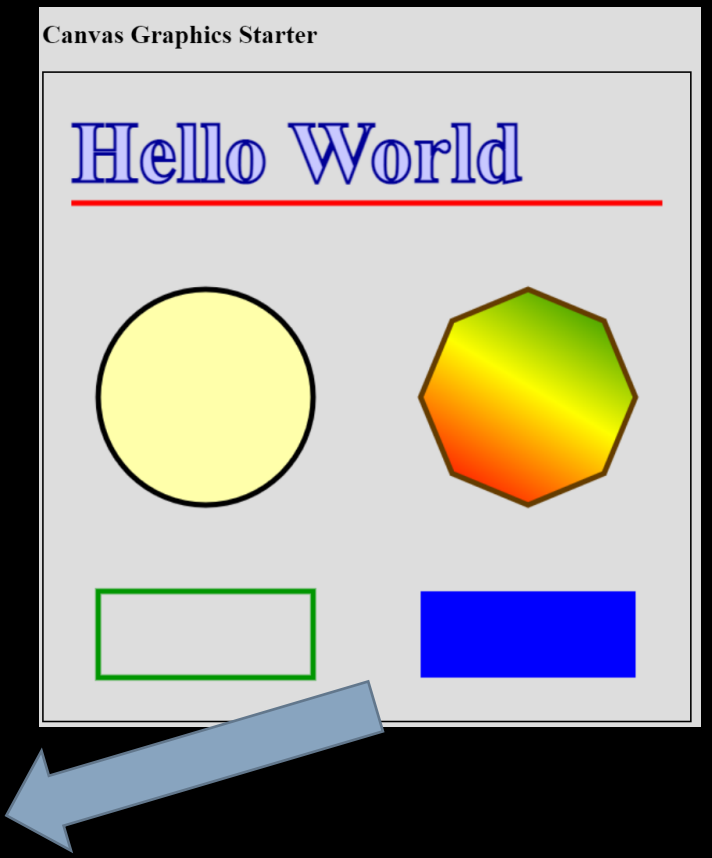





```
38      function draw() {
39
40          // As an example, draw some text and shapes. First, a string of text.
41
42          graphics.font = "bold 80px serif";
43          graphics.fillStyle = "rgb(200,200,255)";  // light blue, as a CSS RGB color
44          graphics.strokeStyle = "rgb(0,0,150)";    // dark blue
45          graphics.lineWidth = 3;
46          graphics.fillText("Hello World", 25, 100);
47          graphics.strokeText("Hello World", 25, 100); // Text can be stroked as well as filled!
```

Sample Canvas 2D Graphics






```
70          // A rectangle can be stroked or filled with one function call:
71
72          graphics.strokeStyle = "rgb(0,150,0)";
73          graphics.strokeRect(50,480,200,80);
74          graphics.fillStyle = "blue";
75          graphics.fillRect(350,480,200,80);
```

Shapes

- A path can be created using functions in the graphics context
- In the current version of the API, paths are not represented by objects, and there is no way to work with more than one path at a time or to keep a copy of a path for later reuse
- Paths can contain lines, Bezier curves, and circular arcs

Shapes

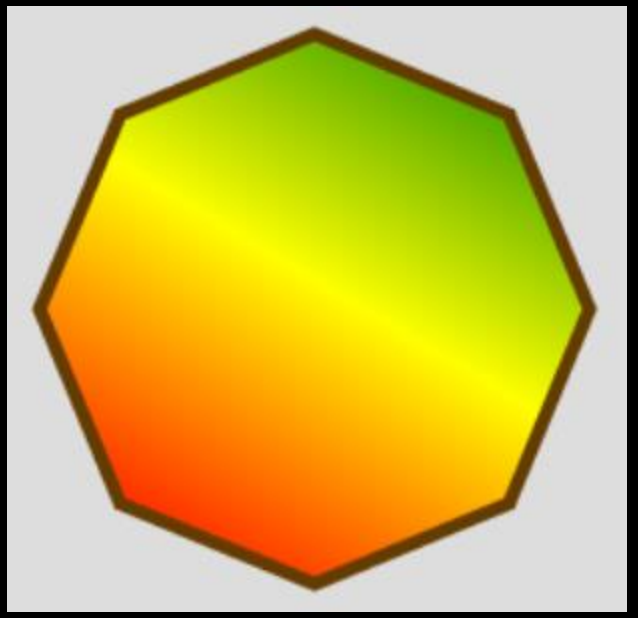
- The following are functions related to paths:
  - graphics.beginPath() – start a new path
  - graphics.moveTo(x,y) – move the current point to (x,y), without adding anything to the path
  - graphics.lineTo(x,y) – add the line segment starting at current point and ending at (x,y) to the path, and move the current point to (x,y)
  - graphics.bezierCurveTo(cx1,cy1,c2x,cy2,x,y) – add a cubic Bezier curve to the path; the curve starts at the current point and ends at (x,y). The points (cx1,cy1) and (cx2,cy2) are the two control points for the curve
  - graphics.quadraticCurveTo(cx,cy,x,y) – adds a quadratic Bezier curve from the current point to (x,y), with control point (cx,cy)
  - graphics.arc(x,y,r,startAngle,endAngle) – adds an arc of the circle with center (x,y) and radius r. The next two parameters give the starting and ending angle of the arc. They are measured in radians. The arc extends in the positive direction from the start angle to the end angle
  - graphics.closePath() – adds to the path a line from the current point back to the starting point of the current segment of the curve
- Creating a curve with these commands does not draw anything
- To get something visible to appear in the image, you must fill or stroke the path

Shapes

```
49          // Stroke a line under the text.  It has to be created as a path.
50
51          graphics.strokeStyle ="red";
52          graphics.lineWidth = 5;
53          graphics.beginPath();
54          graphics.moveTo(25,120);
55          graphics.lineTo(575,120);
56          graphics.stroke();
```

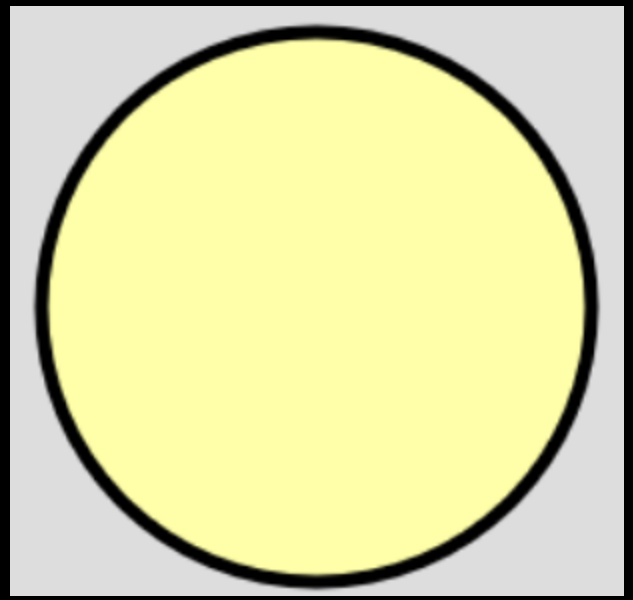


```
77        // Other polygonal shapes can be made as paths.  An octagon, for example:
78
79        graphics.beginPath();
80        graphics.moveTo(550,300);
81        for (let i = 1; i <= 8; i++) {
82            let angle = (2*Math.PI)/8 * i;
83            let x = 450 + 100*Math.cos(angle);
84            let y = 300 + 100*Math.sin(angle);
85            graphics.lineTo( x, y );
86        }
87        graphics.closePath();
88        let gradient = graphics.createLinearGradient(420,420,550,200);
89        gradient.addColorStop(0,"red");
90        gradient.addColorStop(0.5,"yellow");
91        gradient.addColorStop(1,"green");
92        graphics.fillStyle = gradient;  // Use a linear gradient fill!
93        graphics.strokeStyle = "rgb(100,60,0)";
94        graphics.fill();
95        graphics.stroke();
```



Shapes

```
58          // Fill and stroke a circle, which has to be created as a path containing a circular arc.
59
60          graphics.beginPath();
61          graphics.moveTo(250,300); // Move to start point of arc.
62          graphics.arc(150,300,100,0,2*Math.PI);  // A complete circle.
63          graphics.closePath();
64          graphics.fillStyle = "#FFFFAA"; // A CSS hexadecimal color code, light yellow.
65          graphics.strokeStyle = "black";
66          graphics.lineWidth = 5;
67          graphics.fill();
68          graphics.stroke();
```



Transforms

- graphics.scale(sx,sy) – scale by sx in the x-direction and sy in the y-direction.
- graphics.rotate(angle) – rotate by angle radians about the origin. A positive rotation is clockwise in the default coordinate system.
- graphics.translate(tx,ty) – translate by tx in the x-direction and ty in the y-direction.
- graphics.transform(a,b,c,d,e,f) – apply the affine transform x1 = a*x + c*y + e, and y1 = b*x + d*y + f.
- graphics.setTransform(a,b,c,d,e,f) – discard the current transformation, and set the current transformation to be x1 = a*x + c*y + e, and y1 = b*x + d*y + f.

Transforms

- graphics.save() – push a copy of the current state of the graphics context, including the current transformation, onto the stack.
- graphics.restore() – remove the top item from the stack, containing a saved state of the graphics context, and restore the graphics context to that state.

- Using these methods, the basic setup for drawing an object with a modeling transform becomes:

```
graphics.save();           // save a copy of the current state
graphics.translate(a,b);   // apply modeling transformations
graphics.rotate(r);
graphics.scale(sx,sy);
   .
   .  // Draw the object!
   .
graphics.restore();        // restore the saved state
```

Discussions

Launch Assignment 2